# Pointwise Refinements of Worst-Case Computational Complexity

Yunbei Xu
National University of Singapore
yunbei@nus.edu.sg

**Abstract**

Worst-case complexity compresses an algorithm to one number. This note proposes the minimal pointwise refinement: fix the computational model, admissible algorithm class, correctness convention, and resource measure, and keep the full input-wise resource profile of a single worst-case-optimal or near-worst-case-optimal algorithm. The objects are valuable as proof devices for upper-bounding the profile of the fixed algorithm, and their discovery or verification cost must be charged whenever they are produced algorithmically. Optimization complexity fits the same language by taking the profile to be the hitting time to a prescribed accuracy. Parameterized, generic-case, average-case, and smoothed analyses are then recovered as summaries or constraints on profiles. The connection with pointwise statistical dimension is an analogy of form, not an identity of units: both avoid collapsing heterogeneous pointwise behavior too early, but statistical dimension controls risk or sample size, whereas the present object controls computational resources.

## 1 The object

An algorithm has more structure than its worst-case bound. If it solves inputs of length $n$, its computational behavior is the function

$$x \longmapsto \mathrm{Time}(A, x), \qquad x \in \mathcal{X}_n.$$

Worst-case analysis keeps only the scalar

$$\sup_x \mathrm{Time}(A, x).$$

A pointwise refinement keeps the function itself, but with the same minimax quantifier order: the algorithm must be fixed before the input is known. This is the essential point. The refinement is not the best algorithm chosen separately for each input, and it is not yet a representation scheme. A backdoor, tree decomposition, basin of attraction, preconditioner, active set, or search trace may explain why the chosen algorithm is fast on a particular input; it is not part of the definition unless the theorem explicitly uses it, in which case its construction and verification are charged as computational work.

## 2 Computation

Let

$$\mathcal{P} = \{(\mathcal{X}_n, \mathcal{Y}_n, R_n) : n \geq 1\}$$

be a decision or search problem. For $x \in \mathcal{X}_n$, the set $\mathcal{Y}_n(x)$ contains admissible outputs, and $R_n(x, y) = 1$ means that $y$ is correct. Fix a machine model, a resource measure $T$, and an admissible algorithm class $\mathfrak{A}$. This is the usual starting point of computational complexity: the code, advice convention, randomness convention, oracle access, and accounting of resources belong to the model and are fixed before inputs are compared (Blum, 1967; Roughgarden, 2021).

**Definition 2.1** (Global validity and resource profile)**.** *At length $n$, an algorithm $A \in \mathfrak{A}$ is globally valid if it satisfies the stated correctness convention for every $x \in \mathcal{X}_n$. In a randomized model this includes the prescribed success requirement, such as Las Vegas termination or success probability at least $2/3$ uniformly over inputs. Its pointwise resource profile is*

$$p_{A,n}(x) := T(A, x), \qquad x \in \mathcal{X}_n,$$

*with $p_{A,n}(x) = \infty$ if the required termination or correctness convention fails on $x$.*

Let $\mathfrak{A}_n^{\mathrm{val}}$ be the globally valid algorithms at length $n$, and define

$$\mathsf{Prof}_n(\mathfrak{A}) := \{p_{A,n} : A \in \mathfrak{A}_n^{\mathrm{val}}\}.$$

The usual worst-case complexity is

$$W_n(\mathfrak{A}) := \inf_{A \in \mathfrak{A}_n^{\mathrm{val}}} \sup_{x \in \mathcal{X}_n} p_{A,n}(x) = \inf_{p \in \mathsf{Prof}_n(\mathfrak{A})} \|p\|_\infty.$$

**Definition 2.2** (Pointwise refinement of worst-case complexity)**.** *For $\eta \geq 0$, the set of $\eta$-minimax pointwise refinements is*

$$\mathsf{PW}_n^\eta(\mathfrak{A}) := \{p \in \mathsf{Prof}_n(\mathfrak{A}) : \|p\|_\infty \leq W_n(\mathfrak{A}) + \eta\}.$$

*Each element of $\mathsf{PW}_n^\eta(\mathfrak{A})$ is the input-wise resource profile of one globally valid algorithm whose worst-case resource is within $\eta$ of optimal. The scalar worst-case value is recovered by taking a supremum; the pointwise refinement records where that resource is spent.*

This set-valued formulation avoids pretending that a canonical optimal algorithm is always unique. When a theorem selects a particular near-minimax algorithm $A$, the reported pointwise complexity is its profile $p_{A,n}$. When many near-minimax algorithms are relevant, the honest object is the family $\mathsf{PW}_n^\eta(\mathfrak{A})$. The pointwise lower envelope

$$\underline{W}_n(x) := \inf_{A \in \mathfrak{A}_n^{\mathrm{val}}} p_{A,n}(x)$$

can be useful as a diagnostic, but it is not itself a minimax refinement, because the minimizing algorithm may depend on $x$. Indeed, if $\mathcal{X}_n = \{0, 1\}$ and two globally correct algorithms have profiles $(1, M)$ and $(M, 1)$, then $\underline{W}_n(0) = \underline{W}_n(1) = 1$, while $W_n(\mathfrak{A}) = M$. The envelope says that every input is easy for some algorithm; it does not provide one algorithm that is easy everywhere.

## 3 Optimization

Optimization complexity is the same profile language with a different correctness relation. Let $x$ index an objective-oracle pair $(F_x, \mathcal{O}_x)$. A method $A$ produces iterates $z_t$. For a gap functional $\mathrm{Gap}_x$, such as objective suboptimality, stationarity, feasibility residual, or a problem-specific residual, define

$$T_\varepsilon(A, x) := \inf\{t \geq 0 : \mathrm{Gap}_x(z_t) \leq \varepsilon\},$$

with value $\infty$ if the method fails the required success convention. The worst-case optimization complexity is

$$W_n^{\mathrm{opt}}(\varepsilon;\mathfrak{A}) := \inf_{A\in\mathfrak{A}_{n,\varepsilon}^{\mathrm{val}}} \sup_{x\in\mathcal{X}_n} T_\varepsilon(A,x),$$

and the pointwise refinement is the profile $x \mapsto T_\varepsilon(A,x)$ of a single method whose supremum is near this optimum. This is the standard oracle-complexity viewpoint of optimization, but with the entire input-wise profile retained instead of being immediately collapsed to its supremum (Nemirovsky and Yudin, 1983; Nesterov, 2004). Local curvature, error bounds, active sets, and basins of attraction are therefore witnesses for profile upper bounds, not primitive definitions of complexity.

## 4 How existing notions are recovered

The profile view is deliberately conservative: it does not replace classical algorithmic complexity, but refines it. Many beyond-worst-case theories are obtained by applying a functional to the same profile.

Parameterized complexity proves, for one fixed algorithm $A$, bounds of the form

$$p_{A,n}(x) \le f(\kappa(x))\ \mathrm{poly}(n),$$

where $\kappa(x)$ is a specified structural parameter (Downey and Fellows, 2013; Cygan et al., 2015). Average-case complexity studies $\mathbb{E}[p_{A,n}(X)]$ under an input distribution (Levin, 1986; Yao, 1977). Smoothed analysis studies expectations or tails after perturbing an instance (Spielman and Teng, 2004). None of these summaries is the profile itself; each is a way to describe its shape.

Generic-case complexity is recovered as a global density-one specialization. Let $\pi_n$ be reference measures on $\mathcal{X}_n$. For a total algorithm, being fast generically means exactly that, for some time bound $t(n)$,

$$\pi_n\{x \in \mathcal{X}_n : p_{A,n}(x) \le t(n)\} \longrightarrow 1.$$

For a partial generic algorithm, set $p_{A,n}(x) = \infty$ on nonhalting inputs and require that the algorithm never outputs an incorrect answer; the same display is then the density-one halting-and-correctness condition. Thus generic polynomial time is a high-probability statement about a resource profile, not a representation-cell construction (Kapovich et al., 2003; Gilman et al., 2007; Jockusch and Schupp, 2012).

A dimension can also be derived from a profile, but it should be treated as a coordinate system, not as an additional primitive. Choose a nondecreasing computational scale $s_n(d)$, for example $s_n(d) = n^c 2^d$ or $s_n(d) = n^d$. The induced pointwise computational dimension of a profile $p$ is

$$d_{p,n}^{(s)}(x) := \inf\{d \ge 0 : p(x) \le s_n(d)\}.$$

This number simply re-expresses runtime, oracle calls, or iterations in the chosen scale. With $s_n(d) = n^c 2^d$, it measures the exponential overhead beyond an $n^c$ baseline; with $s_n(d) = n^d$, it measures the polynomial exponent. Generic-case complexity is then the statement that this derived dimension is small on a set of $\pi_n$-probability tending to one.

## 5 Connections with pointwise statistical dimension

The analogy with pointwise statistical dimension is clean once the units are kept separate. In statistics, one may assign to a distribution, parameter, learned representation, or local model a pointwise dimension that controls sample size, estimation error, or excess risk. In the present note,

one assigns to an input and a fixed algorithm a pointwise profile, or a derived dimension obtained by inverting a computational scale. Both ideas keep local behavior before taking a global supremum, but they do not measure the same thing.

A statistical statement has the form

$$\mathrm{Risk}(\widehat{f};\mathcal{D}) - \mathrm{Risk}(f^\star;\mathcal{D}) \le r_n\big(d_{\mathrm{stat}}(\mathcal{D}),\delta\big),$$

with probability at least $1-\delta$, whereas a computational statement has the form

$$T(A,x) \le s_n\big(d_{\mathrm{comp}}(x)\big).$$

The first concerns risk, sample size, or information; the second concerns time, memory, oracle calls, iterations, or another specified computational resource. A theorem may relate the two, for example by proving both a generalization bound and a training-time bound for the same learning procedure, but the two profiles should not be merged unless the theorem supplies a common evaluation rule. This is the right way to connect pointwise computation with pointwise statistical dimension and recent pointwise generalization or Gaussian-field viewpoints (Li and Xu, 2026; Xu, 2026): the common principle is pointwise refinement, not equality of statistical and computational quantities.

This also clarifies the role of representation. Statistical pointwise dimension often has a natural local geometry, such as a metric ball, eigenspace, tangent set, or effective rank. Computational instances need not have a canonical metric under which neighboring strings share difficulty. The primary computational object is therefore the resource profile. If an algorithm discovers a decomposition, normal form, local basin, proof state, or search trace, that object is a witness for an upper bound on the profile; it is not required in order to define pointwise computational complexity.

## 6 Conclusion

The pointwise refinement of worst-case complexity is the profile version of a familiar scalar. Fix the algorithm class and resource measure, restrict to globally valid algorithms, and retain the resource profile of a worst-case-optimal or near-worst-case-optimal algorithm. This keeps the standard computational quantifiers while revealing the heterogeneity hidden by the supremum. Optimization complexity fits by replacing correctness with hitting a prescribed residual. Parameterized, generic, average-case, and smoothed analyses are recovered as profile summaries. Pointwise statistical dimension and algorithmic-information complexity are close conceptual relatives, but they belong to different typed criteria.

## Acknowledgements

The author used ChatGPT 5.5 Pro as a research and editorial assistant for literature review, formulation refinement, and exposition. The research proposal and results were developed by the author, who assumes full responsibility for the final form and validity of the work.

## References

Manuel Blum. A machine-independent theory of the complexity of recursive functions. *Journal of the ACM*, 14(2):322–336, 1967.

Marek Cygan, Fedor V. Fomin, Łukasz Kowalik, Daniel Lokshtanov, Dániel Marx, Marcin Pilipczuk, Michał Pilipczuk, and Saket Saurabh. *Parameterized Algorithms*. Springer, 2015.

Rodney G. Downey and Michael R. Fellows. *Fundamentals of Parameterized Complexity*. Springer, 2013.

Robert Gilman, Alexei G. Miasnikov, Alexei D. Myasnikov, and Alexander Ushakov. Report on generic case complexity. *Herald of Omsk University, Special Issue*, 103–110, 2007.

Carl G. Jockusch Jr. and Paul E. Schupp. Generic computability, Turing degrees, and asymptotic density. *Journal of the London Mathematical Society*, 85(2):472–490, 2012.

Ilya Kapovich, Alexei Myasnikov, Paul Schupp, and Vladimir Shpilrain. Generic-case complexity, decision problems in group theory, and random walks. *Journal of Algebra*, 264(2):665–694, 2003.

Leonid A. Levin. Average case complete problems. *SIAM Journal on Computing*, 15(1):285–286, 1986.

Shaojie Li and Yunbei Xu. Pointwise generalization in deep neural networks. arXiv:2605.18598, 2026.

Arkadi S. Nemirovsky and David B. Yudin. *Problem Complexity and Method Efficiency in Optimization*. Wiley, 1983.

Yurii Nesterov. *Introductory Lectures on Convex Optimization: A Basic Course*. Springer, 2004.

Tim Roughgarden, editor. *Beyond the Worst-Case Analysis of Algorithms*. Cambridge University Press, 2021.

Daniel A. Spielman and Shang-Hua Teng. Smoothed analysis of algorithms: Why the simplex algorithm usually takes polynomial time. *Journal of the ACM*, 51(3):385–463, 2004.

Yunbei Xu. Pointwise complexity for Gaussian fields: Upper envelopes, algorithmic lower bounds, and separation. arXiv:2606.07931, 2026.

Andrew Chi-Chih Yao. Probabilistic computations: Toward a unified measure of complexity. In *FOCS*, 1977.